

Graph-Enhanced Transformer for Semi-Supervised Bug Duplicate Detection

Kerem Bayramoğlu Hüseyin Karaca Emirhan Koç
Hasan Erkin Ünlü Rabia Ela Ünlü Çalışkan

Bilkent University

CS 588 – Fall 2025–26

The Duplicate Bug Report Problem



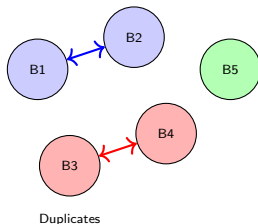
The Challenge

Up to 38% of bug reports are duplicates^[1]

- Eclipse: ~26% duplicates
- Thunderbird: 38% duplicates

Impact

- Wasted developer time
- Delayed issue resolution
- Requires expert knowledge



Goal: Automatically detect duplicates

Why Existing Methods Fall Short



Classical IR (TF-IDF^[3], BM25^[4])

Problem: Vocabulary mismatch^[2]

- Different words, same meaning
- "crash" vs "failure" vs "hang"

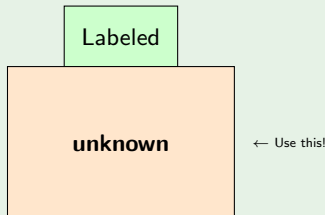
Transformers (BERT^[5], SBERT^[6])

Problem: Label hunger

- Need *many* labeled pairs
- Real repos: mostly unknown

Our Insight

unknown reports contain valuable information!



References: [2] Furnas et al., Vocabulary problem, CACM 1987 — [3] Salton & Buckley, TF-IDF, IPM 1988 — [4] Sun et al., BM25F/REP, ASE 2011 — [5] Devlin et al., BERT, NAACL 2019 — [6] Reimers & Gurevych, SBERT, EMNLP 2019

RQ1: Can unlabeled data be leveraged via graph structure?

- **Challenge:** Most bug reports are unknown; how to use them effectively?
- **Solution:** Build title similarity graph; GNN propagates information during training

RQ2: Can we build a scalable architecture for duplicate bugs?

- **Challenge:** With 50K+ reports, all pairs = $\binom{50000}{2} \approx 1.25$ billion pairs
- **Solution:** Negative sampling

RQ3: Does the approach maintain competitive performance?

- **Challenge:** Strong baselines (BERT, RoBERTa) set high bar
- **Goal:** Novel architecture should achieve competitive results

1 **Semi-supervised learning** leveraging unknown reports (RQ1)

- Title similarity graph connects related issues
- GNN propagates information during training

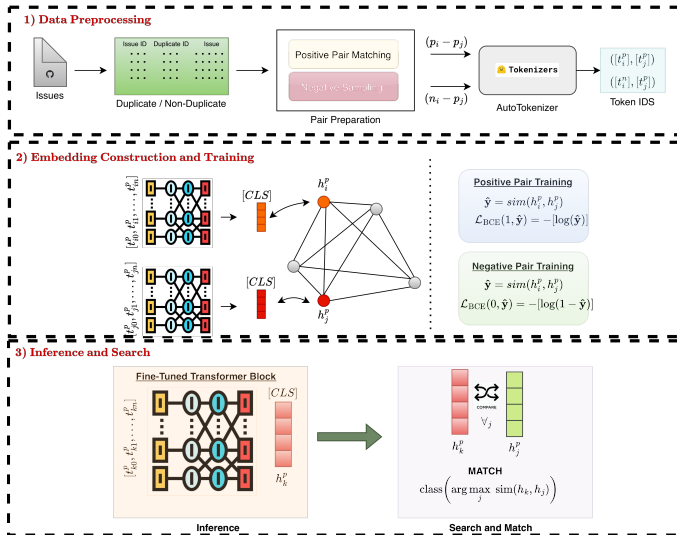
2 **Scalable framework** with negative sampling (RQ2)

- Avoids combinatorial explosion
- GNN-free inference option for deployment

3 **Exceeding performance** against strong baselines (RQ3)

- Novel architecture successfully demonstrated
- Contrastive learning with negative sampling

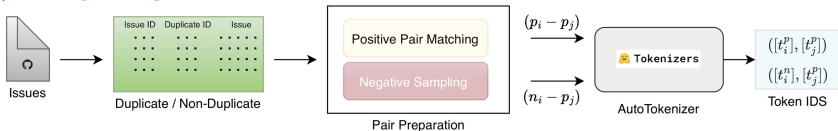
Overall Pipeline



Stage 1: Data Processing



1) Data Preprocessing



Key Steps

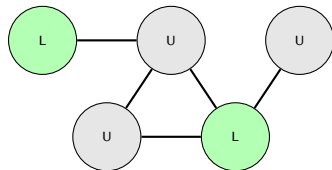
- **Positive pairs:** $\mathcal{P}_i = \{(i, j) \mid g_i = g_j, j \neq i\}$
- **Negative pairs:** $\mathcal{N}_i = \{(i, j) \mid g_i \neq g_j\}$ (50% sampled: $\tilde{\mathcal{N}}_i$)

Stage 2: Graph Construction



Build similarity graph:

- Each issue = one node
- Connect nodes with similar **titles**
- Edge if $\text{sim}(\tau_i, \tau_j) \geq \delta$



L = Labeled

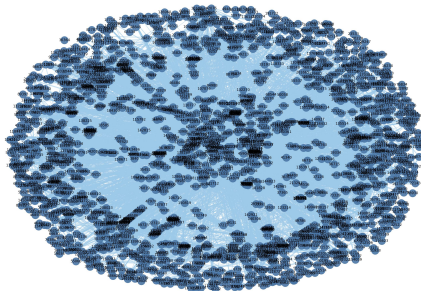
U = unknown

Why?

- Most nodes are **unknown**
- GNN propagates information
- unknown nodes help learning

Graph Structure at Multiple Scales

Portion of graph: eclipse(graph_adj (top_degree, nodes=2000, edges=9245), |e|>=1e-06)



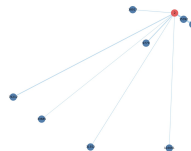
Eclipse: 2000 nodes, 9245 edges

Graph from eclipse(graph_adj (top_degree, nodes=5, edges=10))



5 nodes, 10 edges

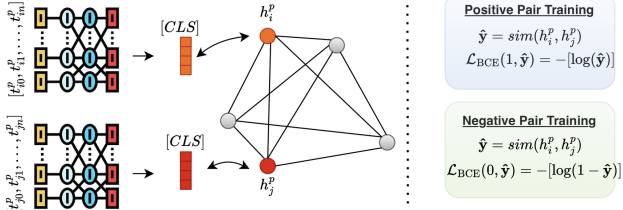
Cap graph for node 2 (top_8)



Node #2 + top-8 neighbors

Stage 2-3: Embedding & Training, Validation and Inference

2) Embedding Construction and Training



Training Strategy

- Jointly train transformer + GNN using graph structure
- Contrastive loss: duplicates $\rightarrow$ close, non-duplicates $\rightarrow$ far
- Total loss: $\mathcal{L} = \sum_{(i,j) \in \mathcal{P}} \mathcal{L}_{ij}^+ + \sum_{(i,j) \in \tilde{\mathcal{N}}} \mathcal{L}_{ij}^-$

Experimental Setup



Baseline Models

- BERT^[5]
- RoBERTa^[10]
- DistilBERT^[11]
- CodeBERT^[12]

Evaluation Metrics

- Precision (P)
- Recall (R)
- F1 Score
- Accuracy

Dataset Statistics^[13]

Project	Total	Non-Dup.	Dup.	% Dup.	Clusters	Pairs
Eclipse	68,124	50,606	17,512	25.7%	6,282	87,224
Thunderbird	26,040	15,994	10,046	38.6%	2,772	53,156

References: [5] Devlin et al., BERT, NAACL 2019 — [10] Liu et al., RoBERTa, arXiv 2019 — [11] Sanh et al., DistilBERT, arXiv 2019 — [12] Feng et al., CodeBERT, EMNLP 2020 — [13] Bettenburg et al., Duplicate Bug Reports, ICSE 2008

Unknown Data Utilization (RQ1)



Validation Set Examples:

Issue1	Issue2	Similarity
<i>Duplicates (Label = 1)</i>		
8470	35054	0.923
70983	83204	0.993
62405	104926	0.979
<i>Non-Duplicates (Label = 0)</i>		
158896	47204	-0.033
124062	21003	-0.107
73654	28863	0.138

Key Insight

Clear separation: Duplicate pairs show high similarity (>0.9), non-duplicates show low similarity (<0.2)

Training & Inference Time Comparison (RQ2)



Table: Training and inference time comparison on Thunderbird dataset

Model	Training (min/epoch)		Inference (ms/query)	
	Time	Rel.	Time	Rel.
BERT ^[5]	16.2	1.0×	12.3	1.0×
RoBERTa ^[10]	16.3	1.01×	12.8	1.04×
DistilBERT ^[11]	15.9	0.98×	8.5	0.69×
CodeBERT ^[11]	16.5	1.02×	8.5	0.69×
Ours	18.1	1.12×	16.6	1.35×

Performance Comparison (RQ3)



Table: Precision, Recall, and F1 scores on Eclipse^[13] and Thunderbird^[13]

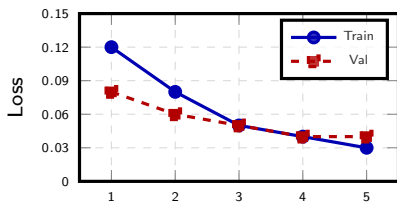
Model	Eclipse			Thunderbird			Avg F1
	P	R	F1	P	R	F1	
BERT ^[5]	0.921	0.933	0.927	0.953	0.961	0.957	0.942
RoBERTa ^[10]	0.918	0.930	0.924	0.946	0.968	0.957	0.941
DistilBERT ^[11]	0.936	0.921	0.929	0.950	0.965	0.957	0.943
CodeBERT ^[12]	0.920	0.928	0.924	0.902	0.930	0.916	0.920
Ours	0.908	0.894	0.901	0.948	0.902	0.924	0.913

References: [5] Devlin et al., BERT, NAACL 2019 — [10] Liu et al., RoBERTa, arXiv 2019 — [11] Sanh et al., DistilBERT, arXiv 2019 — [12] Feng et al., CodeBERT, EMNLP 2020 — [13] Bettenburg et al., Duplicate Bug Reports, ICSE 2008

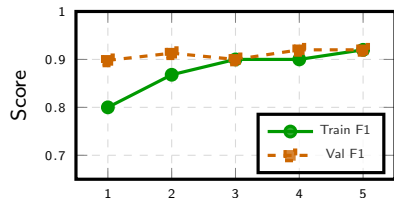
Training Convergence: RoBERTa^[10]



Eclipse^[13]

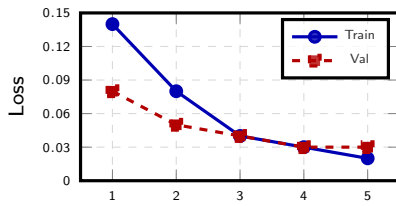


Epoch

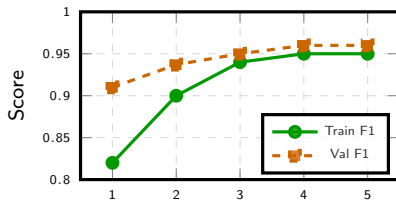


Epoch

Thunderbird^[13]

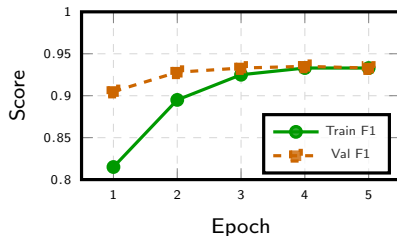
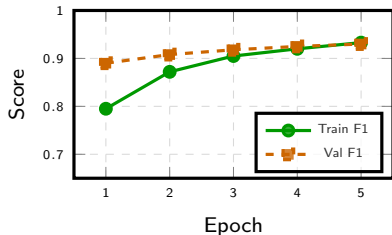
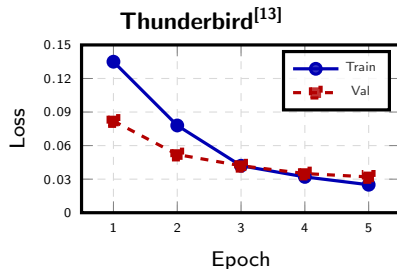
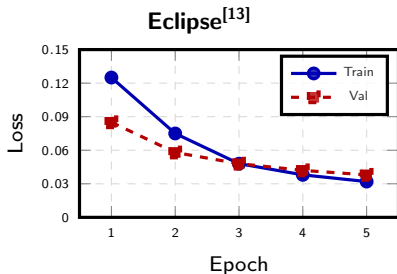


Epoch

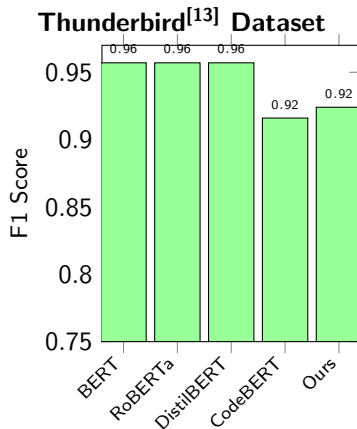
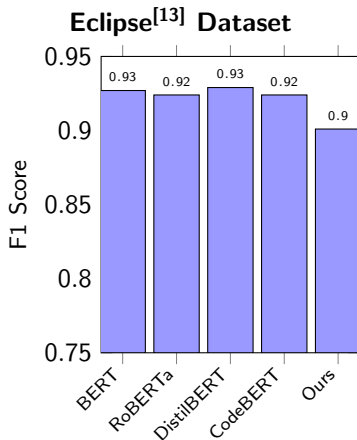


Epoch

Training Convergence: Our Model



Key Results



References: [5] Devlin et al., BERT, NAACL 2019 — [10] Liu et al., RoBERTa, arXiv 2019 — [11] Sanh et al., DistilBERT, arXiv 2019 — [12] Feng et al., CodeBERT, EMNLP 2020 — [13] Bettenburg et al., Duplicate Bug Reports, ICSE 2008

Confusion Matrix – Our Model



Eclipse^[13] Dataset

Threshold = 0.9

Pred: Non-Dup Dup

True: Dup Non-Dup

Non-Dup	8246	765
Dup	892	7542

Acc: 90.50% — Prec: 90.79%
Rec: 89.43% — F1: 90.10%

Thunderbird^[13] Dataset

Threshold = 0.9

Pred: Non-Dup Dup

True: Dup Non-Dup

Non-Dup	2851	258
Dup	512	4694

Acc: 90.74% — Prec: 94.79%
Rec: 90.16% — F1: 92.41%

RQ1: Can unlabeled data be leveraged via graph structure?

- **Result:** ✓ **Unlabeled duplicates map to similar points**, improving graph construction quality

RQ2: Can we build a scalable architecture?

- **Result:** ✓ **Competitive efficiency** — training $1.12\times$ and inference $1.35\times$ slower than BERT baseline

RQ3: Does the approach maintain competitive performance?

- **Result:** ✓ Average F1: **0.913** vs best baseline **0.943** (DistilBERT) — competitive performance, architecture validated

Summary: Novel graph-enhanced transformer for duplicate bug detection

Key Contributions:

- Scalable architecture: Negative sampling + transductive inference (competitive efficiency)
- Semi-supervised learning: Successfully leverages unlabeled reports via similarity graphs
- Competitive performance: Average F1 0.913 vs best baseline 0.943
- All three research questions successfully addressed

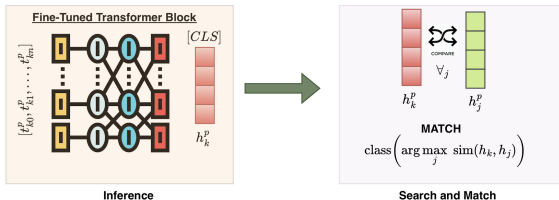
Future Work:

- Multi-modal inputs (vision transformer) hyperparameter tuning, advanced GNNs (GNN-free inductive inference), GraphSAGE

Stage 4*: Future Inference (Optional)



3) Inference and Search



Key Advantage

- **Fast inference:** GNN dropped at test time
- Same speed as SBERT^[6] (no graph overhead)
- Scalable to large repositories

1 High-dimensional embedding space

- Pretrained BERT outputs (768-dim) show very high similarities
- PCA dimensionality reduction (10-dim) dramatically improved separation
- Lack of rigorous experimentation on optimal dimensionality

2 Batch-wise embedding updates

- Current approach: Full node embedding update per batch
- Alternative: Incremental/residual updates for stability

3 Alternative embedding approaches

- LLM-based embeddings could improve semantic similarity
- Trade-off: Practical gains vs. fundamental research rigor

Threats to Validity



- **No cross-validation:** Single train/val/test split used
- **No significance testing:** Statistical tests not performed due to computational budget constraints
- **Limited datasets:** Only Eclipse and Thunderbird evaluated
- **Hyperparameter selection:** Limited search due to resource constraints

Computational Requirements:

- **Hardware:** NVIDIA A100 GPU (Google Colab High Performance)
- **Memory:** ~30GB GPU memory required
- **Batch size:** 64
- **Training time:** 10-20 min/epoch, 5-10 epochs typical

Reproducibility Package:

- **Code:** `github.com/huseyin-karaca/graph-enhanced-dbd`
- **Pretrained models:** BERT-base-uncased checkpoints

- **Emirhan Koç:** Conceptualization, Project Idea, Model Development, Conducting Experiments, Paper Writing.
- **Hüseyin Karaca:** Conceptualization, Code Review, Reproducibility, Reporting, Presentation Design, Paper Writing.
- **Kerem Bayramoğlu:** Conceptualization, Implementation, Reproducibility, Experimental Design, Paper Writing.
- **Hasan Erkin Ünlü:** Conceptualization, Reproducibility, Paper Writing, Presentation Preparation, Literature Survey, Baseline Analysis.
- **Ela Ünlü:** Conceptualization, Reproducibility, Paper Writing, Presentation Preparation, Feedback & Review, Data Visualization, Error Analysis.

Questions?

Thank you for your attention!

GitHub Repository:

`huseyin-karaca/graph-enhanced-dbd`

